

StudioNotebook • Volume 1

What We Are Building

StudioNotebook is a focused workspace for architecture and design students who want a living archive of studio research. It captures categories such as structure, furniture, and materials, along with the specs, dimensions, prices, and image references needed before crits or exams. Version 1.1 already runs end-to-end: users authenticate, create private categories, and later drill into each board to track entries.

Why This App Exists

Studio pin-ups generate a mountain of assets—photos, supplier quotes, dimensional sketches—that rarely survive past a single crit. StudioNotebook keeps those assets in one searchable system so the next semester (or the next boss) can benefit from the same research. It is intentionally production-minded: we treat auth, APIs, and storage the way a small professional team would.

Technology Stack (and Why)

Layer	Tech	Why it fits
Backend API	FastAPI	Async-friendly, declarative typing, automatic docs, ideal for REST-style CRUD services.
Database	SQLite + SQLAlchemy	Zero-admin for local learning, still uses migrations-like models, and can be swapped for Postgres later.
Auth & hashing	bcrypt	Stable hashing algorithm with proper salt handling and industry usage.
Frontend	Vanilla JS + HTML + CSS	Keeps the learning focus on network flows and state management without hiding details inside frameworks.
Static hosting	python -m http.server	Minimal friction for serving the frontend over HTTP so fetch() works without CORS drama.
HTTP server	uvicorn	Standard ASGI runner for FastAPI with reload + WatchFiles for tight dev loops.

Architecture in Practice



- Frontend: Public landing + authenticated workspace, served statically but calling the API for live data.
- Backend: FastAPI routes that validate payloads, issue JWT-style session tokens (stored as UUIDs), and enforce per-user ownership.
- Database: SQLite file that keeps tables for `users`, `session_tokens`, `categories`, `entries`, and `entry_images`.

Data Flow IRL

1. User signs up or logs in from the UI. JS serializes the form and POSTs it to `/signup` or `/login`.
2. FastAPI validates the request, hashes the password with bcrypt, stores the user, and returns a token. That token is persisted in `localStorage` on the frontend.
3. Every authenticated request adds `Authorization: Bearer <token>` so backend dependencies can fetch the `SessionToken`, resolve the user, and scope all queries.
4. When a new category is saved, the frontend POSTs JSON to `/categories`. FastAPI writes a row to SQLite via SQLAlchemy and returns the object. The UI re-fetches to keep cards in sync.
5. File uploads (entry images) go through multipart forms handled by FastAPI, written to `/backend/uploads`, and referenced in the DB.

Frontend ↔ Backend Collaboration

- **Frontend team** would own `frontend/` (HTML, CSS, JS) and talk to the backend via the documented endpoints.
- **Backend team** would evolve `backend/`, the DB schema, and deploy API versions. Separation allows each side to ship independently as long as the contract (JSON payloads + responses) stays consistent.

Development Timeline up to v1.1

1. **Foundations:** Scaffolded FastAPI project, SQLite engine, models for categories & entries, and vanilla HTML shell.
2. **CRUD for categories/entries:** Added list/create/delete endpoints, wired forms, and ensured relational ownership via SQLAlchemy relationships.
3. **File uploads:** Built `/uploads` static mount, integrated `UploadFile` handling, and added image chips in the table.
4. **Authentication (v1.1 milestone):**
 - Implemented `/signup`, `/login`, `/logout`, `/me`.
 - Added `SessionToken` table and helper dependencies `get_current_session` + `get_current_user`.
 - Updated frontend to persist tokens, guard pages, and display nav state.
5. **Post-login UX:** Added workspace hero, stat cards, and CTA flows so the landing page adjusts after auth.

For every step we kept a running reason:

- **Why auth now?** Needed per-user privacy before continuing with sharing/export features.
- **Why tokens over cookies?** Simpler to inspect in `localStorage` for a single-page style app; avoids CSRF complexities at this stage.
- **Why SQLite first?** Low ceremony, but the ORM layer means we can switch to Postgres without rewriting business logic.

Backend Concepts in Context

- **POST vs GET:** GET returns data without changing state (e.g., `/categories`). POST sends data to create or mutate state (e.g., `/signup`, `/categories`). In our API, POST endpoints expect JSON or multipart forms and respond with JSON resources.
- **Frontend sending data:** JavaScript uses `fetch()` with `body: JSON.stringify(payload)` plus `Content-Type: application/json`. For uploads we pass a `FormData` instance without manually

setting headers so the browser manages boundaries.

- **Backend receiving data:** FastAPI parses the body into Pydantic schemas (`SignupRequest`, `CategoryCreate`) or form fields, validates them, and raises `HTTPException` if something is off (wrong token, missing field, duplicate name, etc.).
- **Validation & storage:** After validation we hash passwords, commit ORM objects, and return serializable schemas. Errors bubble up as structured JSON with `detail` so the frontend can show toasts.
- **JavaScript ↔ Python conversation:** `safeFetch()` centralizes `fetch()` calls, injects the bearer token, and gracefully handles 401 responses by clearing local state. On success, the JSON payload is used to mutate the DOM (cards, tables, hero stats).
- **Database fit:** SQLAlchemy sessions persist objects; relationships ensure that deleting a category cascades to entries/images. Session tokens also live in the DB, so logout simply deletes a row.

Issues We Faced (and Fixes)

Issue	What went wrong	Why it happened	Fix
Port conflicts	<code>python -m http.server</code> or <code>uvicorn</code> refused to start because a prior process still held the port.	Servers were relaunched multiple times without shutting down previous instances.	Listed processes via PowerShell and <code>Stop-Process</code> on stray PIDs before restarting both servers.
Server not starting	<code>uvicorn</code> failed on import errors after refactors.	Virtual environment missing dependencies after edits.	Re-installed via <code>pip install -r requirements.txt</code> , confirmed <code>.venv</code> activation, then restarted <code>uvicorn</code> .
CORS / "Network error" toast	Browser showed CORS warnings when hitting <code>http://127.0.0.1:8000</code>	The frontend was opened via <code>file://</code> or the backend crashed before CORS middleware replied.	Served the frontend through <code>python -m http.server 5500</code> , added guard in JS to warn if <code>file://</code> is used, and fixed backend exceptions (see <code>bcrypt</code> item).
<code>bcrypt</code> / <code>passlib</code> crash	<code>/signup</code> returned 500 with <code>ValueError: password cannot be longer than 72 bytes</code> and <code>bcrypt</code> has no attribute <code>__about__</code> .	<code>Passlib</code> 's <code>bcrypt</code> backend clashed with the installed <code>bcrypt</code> version.	Replaced <code>passlib</code> usage with direct <code>bcrypt.hashpw/checkpw</code> and pinned <code>bcrypt==4.1.2</code> in <code>requirements.txt</code> .
Connection problems	<code>fetch</code> requests failed after the backend auto-reloaded.	Auto-reload restarted the server during requests.	Added better toast messaging, retried once the server log showed "Application startup complete."

Learning Takeaways (v1.1)

- Authentication reshapes the entire UI flow, not just the backend: tokens, nav states, and conditional hero sections stay in sync.

- Consistent fetch helpers (`safeFetch`) make it painless to layer concerns like auth headers, toasts, and redirect-on-401 behavior.
- Treating the stack like a real product (versioning, release notes, issue logs) forces disciplined reasoning about “why now” and “what problem does this solve.”
- Even on a solo project, separating frontend and backend folders mirrors how teams ship features—handing each other contracts instead of ad-hoc fixes.

Status: StudioNotebook v1.1 ships with authentication + category creation. Next releases (v1.2+) will evolve from this foundation based on real studio workflows (entry analytics, exporting boards, etc.).